start with code that says “everything is required” and then remove fields that aren’t required using:

```
|> validate_required(fields -- [:non_required_field_name])
```

All of our fields are required, so we don’t remove any fields here. Our schema does have a “name” field on it, but we’re not updating that in this changeset, so we don’t refer to it.

We’ll use this changeset in a moment, but let’s create the SmsMessage changeset next.

Create SmsMessage Changesets

We’re going to define two different changesets for the SmsMessage schema. We’ll use different changesets when we create a new SmsMessage versus when we update an existing SmsMessage. Add these functions to the bottom of the Schema.SmsMessage module:

phone_app/lib/phone_app/conversations/schema/sms_message.ex

```
import Ecto.Changeset

def changeset(attrs) do
  fields = [
    :contact_id, :message_sid, :account_sid, :body,
    :from, :to, :status, :direction
  ]

  %__MODULE__{}
  |> cast(attrs, fields)
  |> validate_required(fields)
  |> unique_constraint([:message_sid])
end

def update_changeset(attrs, struct = %__MODULE__{}) do
  fields = [:status]

  struct
  |> cast(attrs, fields)
  |> validate_required(fields)
end
```

Ecto doesn’t require specific names for the changeset functions—they can be named anything we want. But you’ll often see these function names because they clearly label how the changesets are intended to be used.

Use Our Changesets

We use the changeset functions by passing in input parameters. The return value of the functions is an Ecto.Changeset struct that includes the source data,